SOFTWARE PORTFOLIO

Patient Similarity and Cross-Hospital Transportability

Implementation of an eICU mortality prediction study

Ting Lin
MPH Candidate, University of Michigan

Source code: github.com/tinglin0706-ui/eicu-transportability-portfolio

Research question

How does patient similarity relate to the performance of a mortality prediction model when it is transferred from one hospital to another? I implemented a workflow that connects clinical data preparation, source-hospital evaluation, and representation-based similarity analysis to examine this question.

My role

I constructed and checked the study cohort, aligned patient records with sparse feature arrays, implemented cross-hospital evaluation, and developed patient- and hospital-level similarity analyses. Keith Feldman, PhD, provided research guidance and discussed the design and interpretation with me.

AI coding assistants were used during implementation. I designed the analysis, reviewed the code, and verified the outputs.

From clinical records to hospital-level analysis

Stage	Implementation
Define the cohort	Adult eligibility, patient deduplication, and hospital selection
Prepare clinical features	Event extraction and adapted FIDDLE preprocessing workflow
Evaluate the model	Source-hospital cross-validation and fixed-model external evaluation
Represent patients	Pooled TruncatedSVD and outcome-class cosine similarity
Compare hospitals	Class-balanced summaries and associations with performance drop

This document presents selected implementations and verification evidence. The companion poster, *Patient Similarity and Model Transportability across eICU hospitals*, presents the study findings. Both artifacts come from the same research project.

FIDDLE and the associated extraction workflow are third-party resources. My contribution is the project-specific implementation, adaptation, and integration described here; the preprocessing method itself is credited to Tang and colleagues [1].

01 Cohort construction and data integrity

The mortality task uses the first 48 hours of an ICU stay to predict in-hospital death. I retained adults with an ICU stay of at least 48 hours and a known hospital mortality outcome. Patients observed across multiple hospitals were excluded; one eligible stay was retained per remaining patient. Hospitals needed at least 1,000 eligible patients.

Repeat-stay selection uses discharge year when available and stay ID as a deterministic tie-breaker. It does not establish the first chronological ICU admission. Ages reported as greater than 89 are represented as 90.

Preserving patient identity across processing stages

FIDDLE returns separate feature arrays and patient identifiers. A row-order error could attach the wrong hospital to a feature vector while leaving model fitting apparently successful. The loader therefore uses the FIDDLE ID sequence as the left side of a validated join and checks for missing hospital assignments.

SELECTED IMPLEMENTATION / VALIDATED METADATA ALIGNMENT

```
row_info = fiddle_ids.merge(
    metadata,
    on="ID",
    how="left",
    validate="one_to_one",
    sort=False,
)

if row_info["hospitalid"].isna().any():
    missing_ids = (
        row_info.loc[row_info["hospitalid"].isna(), "ID"].head(10).tolist()
    )
    raise ValueError(
        "Some FIDDLE IDs have no hospital ID in the population file. "
        f"Examples: {missing_ids}"
    )

return row_info
```

Source: pooled_svd_embeddings.py | read_patient_metadata

Clinical event preparation

Extracted events share four fields: patient ID, time from ICU admission, variable name, and value. Preparation retains timed records in the 48-hour observation window and untimed static records, removes identifier-like variables using explicit name patterns, and resolves duplicate patient-time-variable records. Mortality labels and hospital metadata are kept separate from the predictor matrix.

FIDDLE output includes a static matrix and a temporal tensor. I load the PyData-sparse arrays, flatten the temporal dimensions while preserving the patient axis, and concatenate static and temporal predictors. The synthetic check deliberately reorders metadata and feature IDs independently to test this alignment.

02 Source-only fitting and external evaluation

The prediction model uses unreduced FIDDLE features. Source hospital 73 supplies the training records; target hospitals are reserved for external model evaluation. Five stratified source folds provide the internal performance baseline. Within each outer training fold, a separate validation subset is used to choose the F1 threshold.

The excerpt begins after the training, validation, and test subsets have been constructed. Model parameters are fitted on the training subset, the threshold is selected on validation predictions, and evaluation uses the untouched outer test subset.

SELECTED IMPLEMENTATION / SEPARATE FITTING, THRESHOLD SELECTION, AND TESTING

```
model.fit(x_train, y_train)

val_prob = model.predict_proba(x_val)[:, 1]
tuned_threshold = choose_best_f1_threshold(y_val, val_prob)

test_prob = model.predict_proba(x_test)[:, 1]
metrics = calculate_metrics(y_test, test_prob, tuned_threshold)
```

Source: cross_hospital_transportability.py | run_source_hospital_cv

Transfer to target hospitals

After source cross-validation, the model is refitted on all source patients. The mean of the fold-specific thresholds is fixed for every target hospital. Target outcomes are used for performance evaluation, not model fitting or threshold selection. Class weighting is estimated from the relevant source training labels.

Research configuration	Setting
XGBoost	300 estimators; maximum depth 8; learning rate 0.05
Source cross-validation	5 stratified folds; 25% of each outer training fold for validation
Reproducibility	Random seed 42; explicit configuration and saved run settings

Keeping downstream analysis tied to the evaluated model

The portfolio workflow exports individual target predictions from the same refitted model used for external evaluation. Subsequent stages read those predictions, the selected threshold, and the current source CV baselines. This avoids combining similarity analysis with metrics from an unrelated run.

Performance drop is source CV performance minus target performance. Positive values indicate lower external performance. Because source CV models use fewer training records than the final refitted model, this difference also reflects unequal training-set sizes. AUPRC is implemented as average precision and varies with outcome prevalence.

03 Patient similarity and interpretation

A separate TruncatedSVD representation supports similarity analysis. The study configuration retains 500 components. SVD is fitted to pooled features from source and target hospitals; labels and hospital IDs are excluded from its inputs. Target feature distributions nevertheless inform the representation, making this a transductive analysis.

For each target patient, I calculate mean cosine similarity to source decedents and, separately, to source survivors. Processing query patients in blocks avoids constructing one full target-by-reference similarity matrix. The optional self-exclusion branch supports a within-hospital exploratory control.

SELECTED IMPLEMENTATION / CHUNKED COSINE SIMILARITY

```
for start in range(0, n_query, chunk_size):
    stop = min(start + chunk_size, n_query)
    block = cosine_similarity(query_vectors[start:stop], reference_vectors)

    if exclude_self:

        for local_index, global_index in enumerate(range(start, stop)):
            block[local_index, global_index] = np.nan
        means[start:stop] = np.nanmean(block, axis=1)
    else:
        means[start:stop] = block.mean(axis=1)
```

Source: similarity_utils.py | mean_similarity_to_reference

From patient scores to hospital summaries

Let **d(i)** and **s(i)** denote the two mean similarities. The patient score **absolute lean = |d(i) - s(i)|** measures how differently a patient resembles the two source outcome groups. It is not a measure of overall source similarity or a mortality probability.

For each target hospital, I average the score separately among patients who died and patients who survived, then give those two means equal weight. Hospital-level correlations compare this class-balanced summary with performance drop. Outcome-stratified analyses examine the two groups separately.

The distinction between score construction and retrospective analysis matters. Comparing one target patient with source outcome groups does not require that target patient's label. Class-balancing target hospital summaries and defining target outcome groups do require target labels. Those summaries are therefore retrospective diagnostics, not validated tools for forecasting performance before target outcomes become available.

Hospitals are the observations in correlation tests. Spearman tests receive BH correction within defined score families, rather than a single correction across all exploratory analyses. Associations from one source hospital do not establish causation or generalization to other source hospitals.

04 Verification and reproducibility

The synthetic demonstration retains 480 fictional patients from 487 input rows: 80 at the source and 400 across five target hospital groups. Four SVD components and a smaller model reduce runtime. No original eICU records are included.

Check	Evidence
Cohort and labels	Expected exclusions, censored-age coding, and agreement between cohort and mortality labels
Input preparation	Observation-window filtering, static-record retention, duplicate removal, and identifier-name filtering
Cross-stage consistency	Shuffled IDs and metadata; predictions and similarities for all 400 target records
Numerical behavior	Chunked cosine means, self-exclusion, class-balanced aggregation, and BH adjustment checked against independent calculations
Software checks	All 23 command-line entry points load; all Python files compile; both retained figures are generated

Run the demonstration

The package includes dependencies, configuration, and workflow launchers. With Python 3.10 and the listed dependencies installed:

REPRODUCIBLE SYNTHETIC RUN

```
python examples/synthetic_demo.py --outdir demo_output
```

Entry point: examples/synthetic_demo.py

What these checks establish

The demonstration checks software behavior using preconstructed FIDDLE-format arrays. It does not test FIDDLE transformation or extraction from original clinical tables, and it does not establish numerical reproduction of the poster. The dependency file records the local verification environment rather than the historical study environment. Synthetic statistics have no clinical interpretation.

Software attribution

[1] Tang S, Davarmanesh P, Song Y, Koutra D, Sjoding MW, Wiens J. Democratizing EHR analyses with FIDDLE: a flexible data-driven preprocessing pipeline for structured clinical data. *J Am Med Inform Assoc.* 2020;27(12):1921-1934. doi:10.1093/jamia/ocaa139.

Upstream software: github.com/MLD3/FIDDLE
Extraction and population workflow: github.com/MLD3/FIDDLE-experiments. Adapted components are identified in the package's third-party notice.